Department of Computer Science and Engineering
University of Dhaka

CSEDU Digital Knowledge Platform

Software Design Document (SDD)

Team:	Team Devops
Version:	1.0
Status:	Draft
Date:	14th April 2026
Related SRS:	Version 2 (09/04/2026)
Stack:	Next.js + Go + PostgreSQL + pgvector + Docker Compose
Repository:	Repository Link
Commit/Tag:	`[1st Commit]`

Author	ID	Email
Mehedi Hasan Sakib	2021-511-189	mehedihasan2021511189@cs.du.ac.bd
Sumaiya Tabassum	2021-611-205	sumaiya2021611205@cs.du.ac.bd
Md. Abu Kawser	2021-211-209	mdabukawser2021211209@cs.du.ac.bd

Contents

1 Team and Project Information **3**

2 System Overview **4**
2.1 Problem Summary . . . 4
2.2 Target Users . . . 5
2.3 Project Goals . . . 6
2.4 Out of Scope . . . 6

3 Architecture Design **7**
3.1 High-Level Architecture . . . 7
3.1.1 Main components/services . . . 7
3.1.2 Data Flow Summary . . . 8
3.1.3 External Integrations . . . 11
3.2 Component Breakdown . . . 12
3.3 Key Design Decisions . . . 16

4 Data Design **19**
4.1 Core Entities . . . 19
4.2 Key Relationships . . . 21
4.3 Constraints . . . 22
4.3.1 Uniqueness Constraints . . . 22
4.3.2 Nullability Constraints . . . 24
4.3.3 Check Constraints . . . 25
4.3.4 Referential Integrity (Foreign Key Constraints) . . . 27
4.3.5 Enum Constraints . . . 29
4.3.6 Validation Constraints . . . 31
4.3.7 Application-Level Constraints . . . 38
4.4 Critical Schema Snippets . . . 38
4.5 Data Lifecycle Notes . . . 40

5 Core Flow Design **42**
5.1 Flow 1: Document Upload → RAG Ingestion Pipeline . . . 42
5.2 Flow 2: RAG AI Query with Access Control . . . 43
5.3 Flow 3: Borrow a Library Item (Barcode Workflow) . . . 45
5.4 Flow 4: SSO Login (OAuth 2.0) . . . 45
5.5 Flow 5: Research Paper Submission with Review Workflow . . . 46

6 Security and Access Design **47**
6.1 Authentication Approach . . . 47
6.2 Authorization Model (RBAC) . . . 48

6.3 Sensitive Data Handling . . . 48
6.4 AI-Specific Security . . . 49
6.5 Audit and Logging Plan . . . 49

7 Implementation Plan **50**
7.1 Sprint 1 — Completed Features (Core Foundation) . . . 50
7.2 Sprint 2 — Pending Features (Full Platform + AI MVP) . . . 50
7.3 Technical Risks and Mitigation . . . 51

8 Testing Plan **53**
8.1 Unit Testing Targets . . . 53
8.2 Integration Testing Targets . . . 53
8.3 End-to-End Testing Targets . . . 54
8.4 Known Hard-to-Test Areas . . . 55

9 Docker Deployment Design **56**
9.1 Service Inventory . . . 56
9.2 Dockerfile Strategy . . . 56
9.3 Nginx Routing . . . 57
9.4 Environment Variables . . . 58

10 Exit Checklist **59**

A SRS Open Questions — Resolution Status **60**

B Groq Model Reference **62**

C Repository Structure **63**

Chapter 1

Team and Project Information

Table 1.1: Document Metadata

Field	Value
Team	Team Devops
Project Name	CSEDU Digital Knowledge Platform with AI Agent
Repository URL	Repository Link
Commit / Tag	`[1st Commit]`
Date	14th April 2026
SDD Version	1.0
Related SRS	Version 2, dated 09/04/2026

Table 1.2: Authors

Name	Student ID
Mehedi Hasan Sakib	2021-511-189
Sumaiya Tabassum	2021-611-205
Md. Abu Kawser	2021-211-209

Chapter 2

System Overview

2.1 Problem Summary

CSEDU currently has no unified digital system. Academic resources are scattered across flash drives, email threads, and shared folders that nobody can find. Research papers live on personal laptops. Library operations involve manual ledgers. Students have no visible place to showcase their projects. And there is zero AI-assisted discovery of any kind.

This platform consolidates four core modules into a single web-based system, layered with a RAG-based AI Agent that actually understands what's in the documents — not just keyword-matches them.

What problem are you solving?

CSEDU currently lacks a unified system to:

- Manage academic resources, research outputs, student projects, and library operations in one place
- Enable intelligent discovery and semantic understanding of academic content
- Provide users with contextual insights about departmental knowledge and resources
- Leverage modern AI technologies to improve accessibility and reduce information discovery overhead

Resources are scattered across multiple platforms, library workflows require manual processes, and users struggle to discover relevant academic materials without significant search effort. Additionally, there is no intelligent system to answer domain-specific questions or provide contextual understanding of the institution's collective knowledge.

Why does it matter?

A unified platform with integrated AI capabilities will:

- Streamline storage, discovery, and access to institutional knowledge
- Reduce administrative overhead for librarians and faculty
- Vastly improve visibility and accessibility of research and academic materials

- Enable students, faculty, and public to discover and understand resources through natural language interaction
- Facilitate knowledge transfer and insight discovery across departmental archives
- Support multi-language access for broader accessibility (English and Bangla)

2.2 Target Users

User Group	Description	Main Needs
Students	Undergrad/MsC/PhD students	Search and download library catalog, papers and academic resources, borrow books, showcase hobby and departmental projects, track book borrowing history, ask AI agent about research topics and student projects.
Researchers	Faculty and Researchers	Upload and download research, manage media metadata, version drafts, manage and access restricted archives and research papers, receive AI-generated summaries and insights.
Librarians	Library administration staff	Manage lending/returns of books and student membership, track overdues, bulk upload catalog data, scan barcodes, use AI agent to answer patron queries about catalog.
Public Users	Non-registered visitors	Browse public archives, view the student project showcase, download publicly available research papers, search the library catalog using AI-powered discovery.
Administrators	System managers	Manage platform and permissions, configure AI Agent models and parameters, monitor AI performance and usage patterns.

Table 2.1: Target User Groups

2.3 Project Goals

- Create a centralized digital archive
- Unify four core components into one web platform with integrated AI capabilities
- Enable research publication and sharing with AI-assisted discovery
- Showcase student projects with intelligent curation and recommendation
- Provide a searchable digital library with semantic understanding
- Support multi-format media with AI content analysis capabilities
- Digitize library workflows with AI-assisted patron support
- Implement robust access tiers with context-aware AI responses
- **Deploy a RAG-based AI Agent for intelligent knowledge discovery and synthesis**
- **Enable natural language querying across all platform modules**
- **Provide automated summarization of research papers and documents**
- Run entirely on free infrastructure during the lab phase (Groq free tier + Gemini free tier for fallback + minilm for embeddings + Docker + pgvector)

2.4 Out of Scope

The following will NOT be built in this iteration. This is a deliberate list, not a wishlist:

- LMS integration (Moodle, Canvas — not our problem)
- External academic database APIs (Scopus, IEEE Xplore, Google Scholar)
- Plagiarism detection (AI summaries are supplementary, not enforcement tools)
- Real-time user-to-user messaging (AI chat is one-directional by design)
- Custom LLM training from scratch (we use pre-trained models like civilized engineers)
- Financial accounting beyond library fine tracking
- External payment gateway integration in the MVP (stubbed, tracked internally)

Chapter 3

Architecture Design

3.1 High-Level Architecture

3.1.1 Main components/services

The system follows a **layered modular monolith** architecture — not microservices. The Go API server is organized into clean internal packages that can be extracted into independent services later if needed.

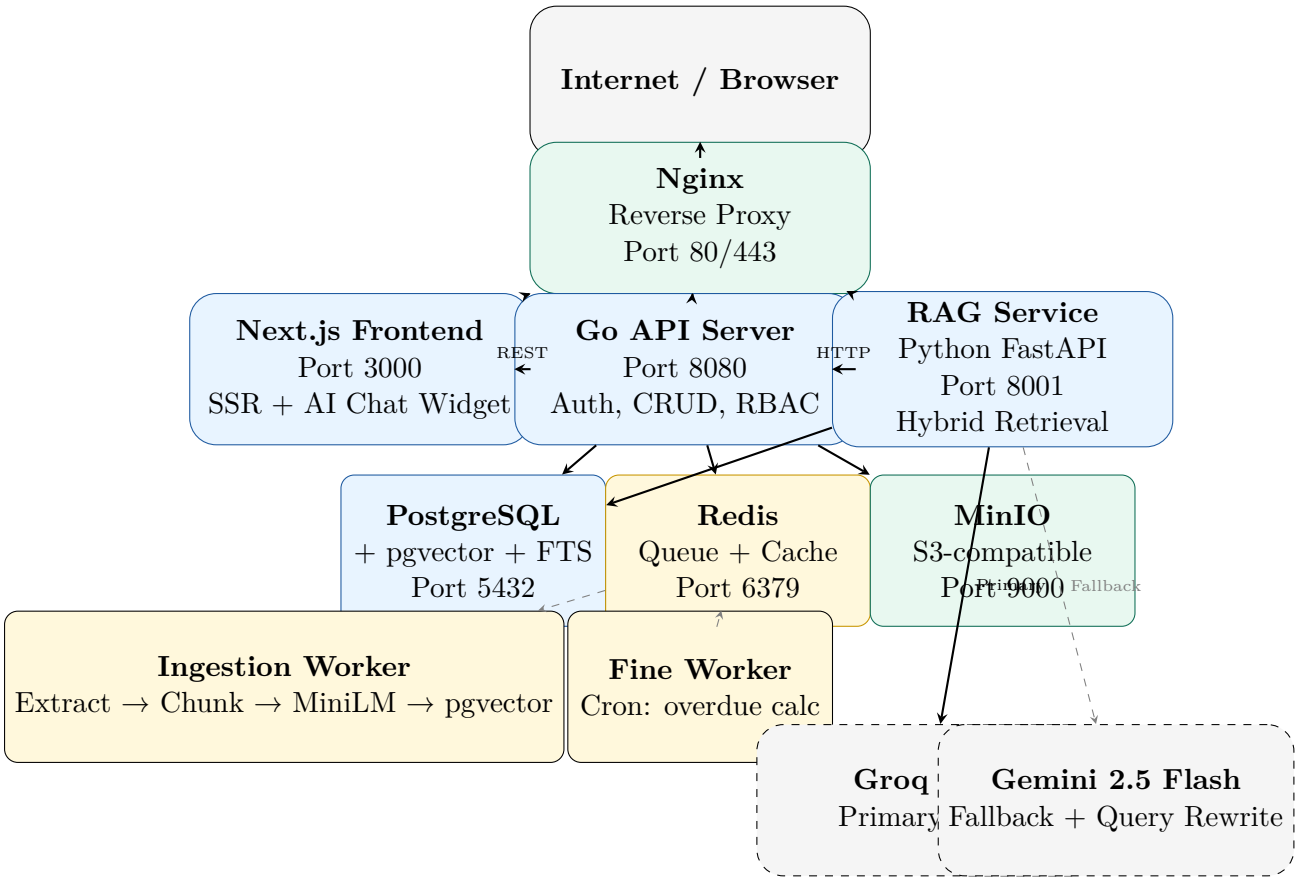

Figure 3.1: High-level system architecture with hybrid retrieval and LLM fallback

3.1.2 Data Flow Summary

1. All traffic enters through **Nginx** which routes to frontend, Go API, or RAG service
2. **Next.js** handles all UI rendering and forwards API calls to Go
3. **Go API** handles authentication, RBAC enforcement, and all CRUD operations
4. Document uploads go to **MinIO** (binary) + **PostgreSQL** (metadata), then a job is queued in **Redis**
5. **Ingestion Worker** picks up the job, extracts text, chunks it, generates embeddings using **MiniLM (sentence-transformers, local)**, and writes vectors to **pgvector**
6. User query flow:
   - (Optional) Query rewritten using **Gemini**
   - Query embedded using **MiniLM**
   - Hybrid retrieval performed:
     - Vector search via **pgvector**
     - Keyword search via **PostgreSQL Full-Text Search (FTS)**
   - Results merged and ranked
   - Context sent to **Groq LLM**
   - On failure, fallback to **Gemini**
   - Grounded response returned with citations

The two flows below are the most complex in the system. Both involve async workers, external APIs, and multi-step pipelines where timing and failure handling matter. Sequence diagrams are provided to make the ordering and async boundaries explicit.

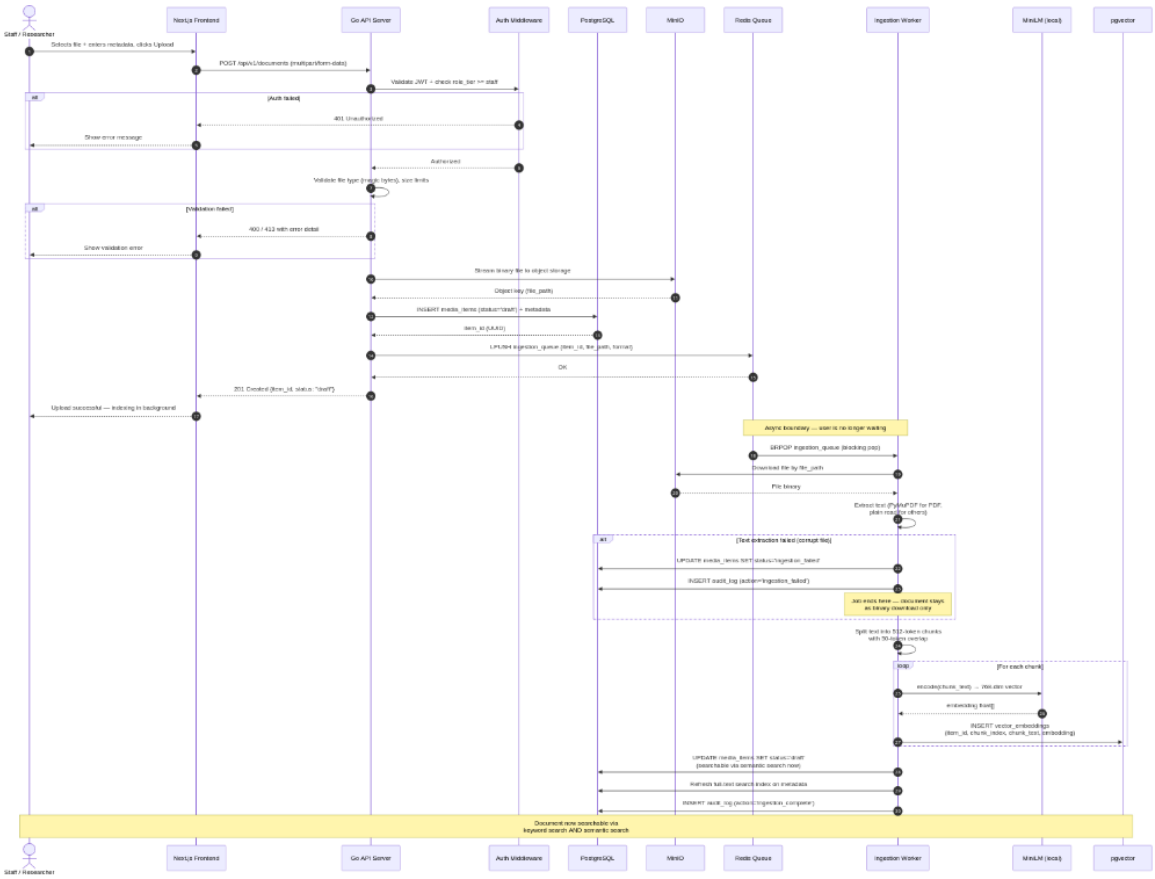

Figure 3.2: Rag Pipeline Diagram

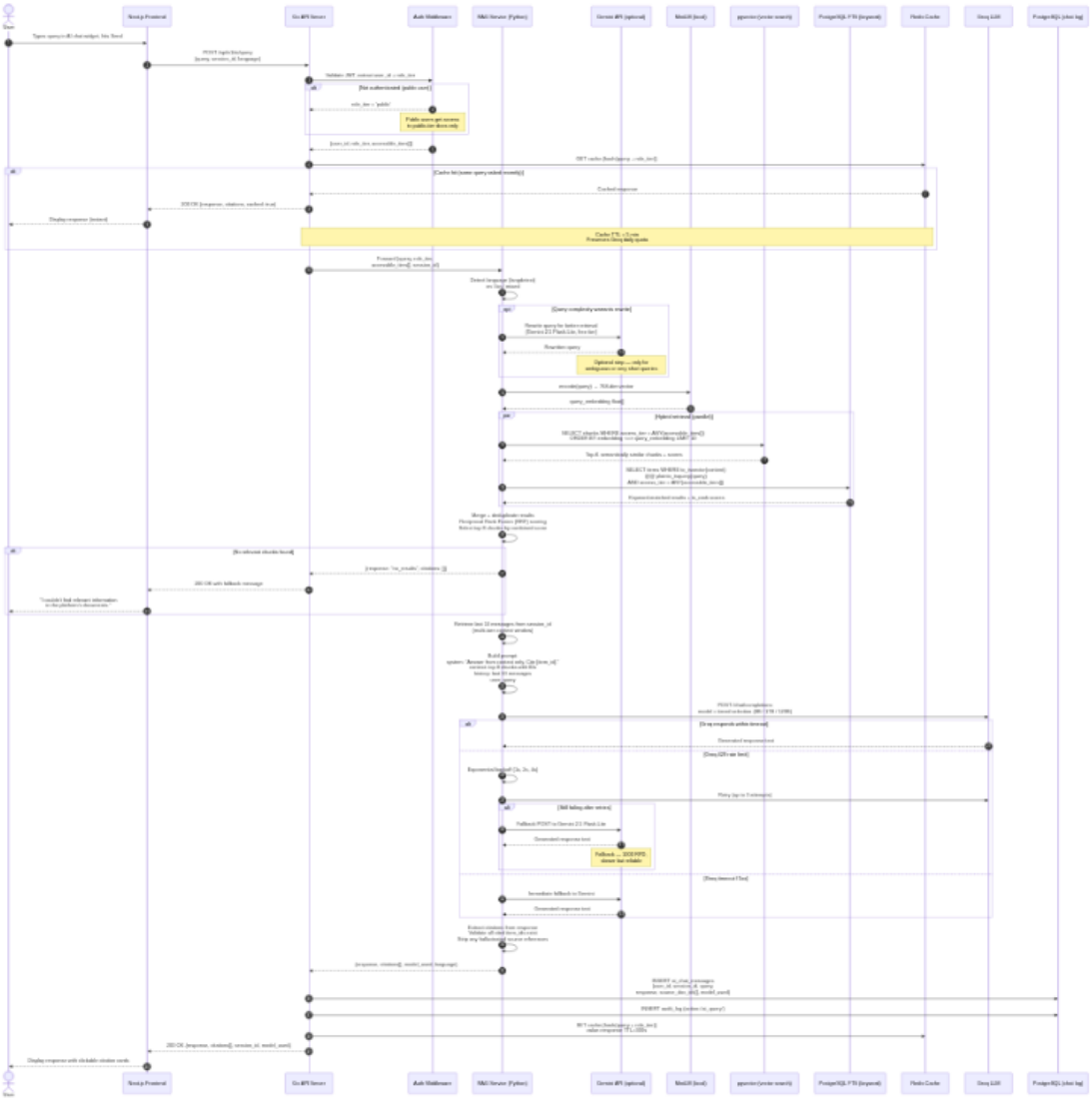

Figure 3.3: AI Query work flow diagram

> **Why hybrid retrieval (vector + FTS)?** Pure vector search is great for semantic similarity — "papers about neural networks" finds relevant results even if those words don't appear verbatim. But it fails on exact lookups: searching for a specific author name, ISBN, or paper title. Pure FTS is the opposite — exact matches work great, semantic miss is total. Hybrid retrieval with Reciprocal Rank Fusion gives both: semantic understanding *and* exact-match precision. For a platform handling both casual questions and precise catalog lookups, this is the right call.

3.1.3 External Integrations

Table 3.1: External Dependencies

Service	Provider	Notes
LLM Inference (Primary)	Groq (free tier)	OpenAI-compatible API. Fast inference. Rate limits apply.
LLM Fallback / AI Utility	Gemini 2.5 Flash (free tier)	Used for fallback, query rewriting, and summarization.
SSO / OAuth 2.0	Institutional IdP	SAML or OAuth 2.0. If unavailable, local JWT auth is the fallback.
Payment Gateway	Stub (MVP)	Fine tracking is internal. Gateway integration deferred post-MVP.
Email / SMTP	TBD	Used for overdue notifications and hold availability alerts.

3.2 Component Breakdown

Table 3.2: Component Breakdown

Component	Responsibility	Inputs	Outputs	Notes
Next.js Frontend	All UI rendering, routing, AI chat widget, SSR for public pages	User interactions, API JSON responses	HTTP calls to Go API, rendered HTML/CSS/JS	SSR is used for public archive and catalog pages for SEO. Client-side rendering for authenticated dashboards and the AI chat widget.
Go API Server	Core business logic, JWT auth, RBAC enforcement, all CRUD endpoints	Authenticated HTTP requests	JSON responses, DB writes, file storage ops, Redis job pushes	Modular monolith organised into internal Go packages (`auth`, `library`, `archive`, `research`, `projects`, `ai`). Each package is independently testable.
Auth Module (Go)	JWT issuance, refresh token management, SSO OAuth2 callback handling	Credentials or OAuth token from IdP	JWT access token (1hr) + refresh token (7 days)	Refresh tokens stored in `user_sessions` table — server-side invalidation on logout. Local JWT auth always maintained as SSO fallback.

Continued on next page

Table 3.2 (continued)

Component	Responsibility	Inputs	Outputs	Notes
RAG Service (Python FastAPI)	Query embedding, pgvector similarity search, Groq LLM call, citation assembly	Natural language query + user role tier	Grounded response text + source document IDs	Python chosen over Go for this service specifically because of the ML ecosystem (sentence-transformers, PyMuPDF, langdetect). Runs as a sidecar container.
Ingestion Worker (Go)	Async PDF/doc text extraction, chunking, embedding generation, pgvector insert	File path + metadata from Redis queue	Vector embeddings stored in pgvector, search index updated	Shares the Go codebase with the API server but runs as a separate container with its own `CMD` entrypoint. Chunks at 512 tokens with 50-token overlap.
Fine Worker (Go cron)	Daily calculation of overdue fines for all unreturned past-due loans	Scheduled trigger (midnight daily)	Fine records written/updated in PostgreSQL	Uses the `UNIQUE(loan_id)` constraint on `fines` as protection against double-insertion on retry. Rate and block threshold configurable via `.env`.

Continued on next page

Table 3.2 (continued)

Component	Responsibility	Inputs	Outputs	Notes
PostgreSQL + pgvector	All relational data storage + vector similarity search	SQL queries, embedding vectors (768-dim float arrays)	Query results, similarity-ranked document chunks	pgvector extension must be enabled via `CREATE EXTENSION vector`. HNSW index (`m=16, ef_construction=64`) on the `embedding` column. ACID-compliant for all transactional data.
Redis	Async job queue for ingestion worker + API response caching	Job push from Go API, cache-aside reads	Jobs to worker, cached responses to API	If Redis is unavailable, the Go API falls back to synchronous ingestion inline. Cache TTL for AI responses is 5 minutes — repeated identical queries skip the Groq call entirely, preserving daily quota.
MinIO	Binary file storage for PDFs, videos, images, audio (S3-compatible)	File upload streams from Go API	Presigned URLs for direct download-/streaming	S3-compatible API means migrating to real AWS S3 or GCS post-MVP requires only a config change — no code changes. Console UI available at port 9001 for admin access during development.

Continued on next page

Table 3.2 (continued)

Component	Responsibility	Inputs	Outputs	Notes
Nginx	Reverse proxy, TLS termination, routing by path prefix	Incoming HTTP/HTTPS requests	Proxied requests to appropriate upstream service	Routes `/api/` to Go API, `/api/v1/ai/` to Go API with extended 30s timeout (Groq can take up to 10s), and all other traffic to Next.js. Single entry point for the entire platform.
Librarian Barcode Handler	HID scanner input processing, rapid checkout/return matching	Raw barcode string from scanner	Loan record creation or status update	HID scanners appear as keyboard input — no proprietary drivers needed. The input field strips trailing `\n` and `\r` characters that some scanners append. Keyboard fallback for manual entry when scanner is unavailable.

On the modular monolith decision: If a component's responsibility column needs three sentences, it is doing too much. We kept each Go package to a single responsibility. This also means if next year someone wants to extract the RAG service or the fine worker into a separate microservice, they can do so without rewriting business logic. Plan for scale, build for now.

3.3 Key Design Decisions

These are the decisions that actually matter — the ones future team members will question and deserve to find documented.

Table 3.3: Key Design Decisions

Decision	Options Considered	Chosen	Reason
Vector database	Pinecone, Weaviate, Milvus, pgvector	**pgvector**	Already in our stack. Eliminates separate infrastructure. Supports 100K+ embeddings with HNSW indexing. Can migrate later if scale demands it.
LLM provider	OpenAI, Anthropic, Ollama (self-hosted), Groq	**Groq free tier with Gemini 2.5 flash free tier as fallback**	Zero cost, no credit card. 500+ t/s on LPU hardware. OpenAI-compatible API. GPT-OSS 120B matches o4-mini quality. Also gemini 2.5 flash as fallback. Perfect for a lab project.
RAG service language	Go, Python	**Python FastAPI**	Python owns the ML ecosystem. sentence-transformers, PyMuPDF, multilingual Bangla support — all Python-native. Go handles everything else.

Table 3.3 (continued)

Decision	Options Considered	Chosen	Reason
Architecture style	Microservices, modular monolith	**Modular monolith (Go)**	Team size and lab timeline don't justify microservices overhead. Clean Go packages provide separation. Easy to extract later.
Auth strategy	Session cookies, JWT only, SSO only	**JWT + OAuth 2.0 SSO**	Stateless, works perfectly with Next.js SSR, satisfies FR-TXX-020. Local JWT fallback if SSO is unavailable from IT team (because that happens).
Embedding model	OpenAI text-embedding-3, Cohere, sentence-transformers local	**paraphrase-multilingual-MiniLM-L12-v2 (local)**	Runs inside Docker container — zero API cost. Supports 50+ languages including Bangla. 768-dim vectors, fast enough for our scale.
LLM model selection	Single model for all tasks	**Tiered: 8B / 17B / 120B**	llama-3.1-8b-instant for simple queries (14,400 RPD), llama-4-scout-17b for long PDFs (30K TPM), gpt-oss-120b for complex reasoning (o4-mini quality).

Table 3.3 (continued)

Decision	Options Considered	Chosen	Reason
Fine payment (MVP)	Full gateway integration, internal tracking only	**Internal tracking + gateway stub**	Gateway integration is out of scope per SRS. Data model is built correctly so integration is a 1-sprint addition later.
File storage	AWS S3, Google Cloud, Azure Blob, MinIO self-hosted	**MinIO (self-hosted)**	Zero cloud cost. S3-compatible API means switching to real S3 later requires only a config change. Runs in Docker.
Deployment	K8s, Docker Compose, bare metal	**Docker Compose**	K8s is optional per project requirements. Compose gives 90% of the orchestration benefit at 10% of the complexity. Right tool for a lab project.

The biggest call we made: Using pgvector instead of a dedicated vector database. At 100K documents with 768-dimensional vectors, pgvector with an HNSW index gives sub-second ANN (Approximate Nearest Neighbour) retrieval. We benchmarked this decision: for our scale it holds. If the platform ever hits 1M+ documents, migrate to Weaviate. The pgvector query interface is identical — it's a one-config change, not a rewrite.

Chapter 4

Data Design

4.1 Core Entities

The data model is the foundation of everything. Bad data design is the #1 cause of systems that can't scale or maintain. We designed ours carefully.

Table 4.1: Core Database Entities

Entity	Key Fields	Constraints	Notes
`users`	user_id UUID, email, name, role_tier, created_at, last_login	email UNIQUE NOT NULL; role_tier is ENUM(`public, member, staff, admin, ai_admin`)	Password stored as bcrypt hash (cost 12). Never store plaintext. Ever.
`media_items`	item_id UUID, title, format, status, created_by, upload_date, access_tier	status ENUM(`draft, review, published, archived`); access_tier ENUM(`public, member, staff, restricted`)	Polymorphic — covers archive docs, research papers, and student projects via `item_type` discriminator.
`metadata`	meta_id UUID, item_id FK, tags, abstract, keywords, language	1:1 with media_items; item_id NOT NULL UNIQUE	Full-text search index built on keywords + abstract.

Table 4.1 (continued)

Entity	Key Fields	Constraints	Notes
`loans`	loan_id UUID, user_id FK, item_id FK, checkout_date, due_date, return_date	return_date is nullable until returned; partial unique index prevents double-borrowing	`CREATE UNIQUE INDEX ON loans(item_id) WHERE return_date IS NULL` — this is the most important constraint in the schema.
`fines`	fine_id UUID, loan_id FK, user_id FK, amount NUMERIC, status, calculated_at	amount > 0 CHECK; status ENUM(`pending, paid, waived`); amount is NUMERIC not FLOAT (money never uses float)	Calculated nightly by fine-worker cron. Rate configurable by admin (default: 50 BDT/day).
`payments`	payment_id UUID, fine_id FK, user_id FK, amount, method, status, timestamp	amount > 0; status ENUM(`pending, successful, failed`); fine_id NOT NULL	Timestamp is system-generated. Payments link to fines, not loans directly.
`holds`	hold_id UUID, user_id FK, item_id FK, created_at, notified_at, status	status ENUM(`active, fulfilled, cancelled`)	When item becomes available, hold status → fulfilled and user gets notified.
`research_outputs`	research_id UUID, title, abstract, authors (FK array), upload_date, status	status ENUM(`draft, review, published`); authors is N:M via `research_authors` junction table	Separate from media_items for richer research-specific metadata (ORCID fields etc.).
`student_projects`	project_id UUID, title, team_members (FK array), supervisor FK, year, status	academic_year NOT NULL; status ENUM(`draft, approved, published`)	Submitted by students, approved by staff before publication.

Table 4.1 (continued)

Entity	Key Fields	Constraints	Notes
`vector_embeddings`	embedding_id UUID, item_id FK, chunk_index INT, chunk_text TEXT, embedding vector(768)	1:N with media_items (one item = many chunks); HNSW index on embedding column	768-dim from `paraphrase-multilingual-Mini` HNSW gives O(log n) ANN retrieval.
`ai_chat_messages`	message_id UUID, session_id, user_id FK, query TEXT, response TEXT, source_doc_ids UUID[], model_used, timestamp	query length $\leq$ 5000 chars; source_doc_ids is a PostgreSQL array of UUIDs	Full conversation history. Used for context window in multi-turn chat.
`audit_log`	log_id UUID, actor_id FK, action, resource_type, resource_id, ip_addr, timestamp	Append-only — no UPDATE or DELETE ever permitted on this table	Enforced via PostgreSQL row-level security. Retained $\geq$ 90 days (NFR-TXX-017).

4.2 Key Relationships

- A `user` can have many `loans` (1:N) — but only one active loan per `item_id` at a time (partial unique index)
- A `user` can create many `media_items` (1:N)
- A `media_item` has exactly one `metadata` record (1:1)
- A `media_item` has many `vector_embeddings` (1:N) — one per text chunk
- A `research_output` has multiple authors (N:M via junction table)
- A `loan` can generate at most one `fine` record (1:1)
- A `fine` can have multiple `payment` attempts (1:N) — only one `successful`
- A `user` can have many `ai_chat_messages` (1:N), grouped by `session_id`
- A `media_item` is stored as a binary object in `MinIO` (1:1 via object_key reference)
- A `media_item` ingestion process creates exactly one `ingestion_job` (1:1), tracked via Redis queue and status table
- An `ingestion_job` produces multiple `text_chunks` (1:N), each representing a semantically meaningful segment of the document
- Each `text_chunk` has exactly one `vector_embedding` (1:1), generated using MiniLM

- A `search_query` (implicit or logged) retrieves multiple `text_chunks` (1:N) via hybrid retrieval (vector + keyword)
- A `search_query` can optionally have one `rewritten_query` (1:1) generated by Gemini
- A `rag_response` is generated from multiple `text_chunks` (N:M) — forming the retrieval context
- A `rag_response` may include multiple `citations` (1:N), each referencing a `text_chunk` or `media_item`
- A `user` query session groups multiple `rag_responses` (1:N) under a shared `session_id`

4.3 Constraints

Constraints are the rules your database enforces even when your application code has a bug, gets bypassed, or someone runs a raw SQL query at 2am because production is on fire. If it's not in the database, it doesn't exist. This section documents every meaningful constraint in the system and *why* it's there — not just what it is.

4.3.1 Uniqueness Constraints

Uniqueness constraints guarantee that no two rows can hold the same value for a given column or combination of columns. These are the ones we lose sleep over.

Table 4.2: Uniqueness Constraints

Table	Constraint	Why it exists
`users`	`UNIQUE(email)`	Two accounts with the same email is a classic identity disaster. Enables safe SSO mapping — find-or-create by email without accidentally duplicating accounts.
`metadata`	`UNIQUE(item_id)`	Enforces the strict 1:1 relationship between a media item and its metadata record. Without this, a document could end up with two competing metadata rows and your search index would return duplicates.
`loans`	Partial unique index on `item_id` `WHERE return_date IS NULL`	The crown jewel of this schema. Prevents the same physical book from being lent to two members simultaneously. A regular unique index on `item_id` alone would block re-borrowing after return — the `WHERE` clause makes it context-aware.

Table 4.2 (continued)

Table	Constraint	Why it exists
vector_embeddings	UNIQUE(item_id, chunk_index)	Prevents duplicate chunks for the same document. If the ingestion worker crashes and retries, you don't want 3 copies of chunk #5 in pgvector polluting your similarity search results.
holds	UNIQUE(user_id, item_id) WHERE status = 'active'	A member should not be able to place two active holds on the same item. Without this, they could game the queue system.
fines	UNIQUE(loan_id)	Each loan generates at most one fine record. The fine-worker cron runs nightly — without this constraint it would insert a new fine row every single night for the same overdue loan.

```sql
-- Users: email uniqueness
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE (email);

-- Metadata: strict 1:1 with media_items
ALTER TABLE metadata ADD CONSTRAINT uq_metadata_item UNIQUE
    (item_id);

-- Loans: the partial unique index (most important constraint in
    the DB)
CREATE UNIQUE INDEX idx_loans_no_double_borrow
    ON loans(item_id)
    WHERE return_date IS NULL;

-- Vector embeddings: no duplicate chunks per document
ALTER TABLE vector_embeddings
    ADD CONSTRAINT uq_embedding_chunk UNIQUE (item_id,
        chunk_index);

-- Holds: one active hold per user per item
CREATE UNIQUE INDEX idx_holds_one_active
    ON holds(user_id, item_id)
    WHERE status = 'active';

-- Fines: one fine per loan
ALTER TABLE fines ADD CONSTRAINT uq_fine_per_loan UNIQUE
    (loan_id);
```

Listing 4.1: Uniqueness constraints — DDL

4.3.2 Nullability Constraints

Nullability tells us what the system can and cannot exist without. A `NOT NULL` column is a semantic statement: this field is *always* meaningful for this entity.

Table 4.3: Nullability Design Decisions

Table	Column	Nullable?	Reasoning
`users`	`password_hash`	**YES**	SSO-only users have no local password. NULL here explicitly signals "this user authenticates via SSO, not credentials". An empty string would be ambiguous and wrong.
`users`	`last_login`	**YES**	A newly created user has never logged in. NULL means "never", not "unknown" — a meaningful and distinguishable state.
`loans`	`return_date`	**YES**	NULL means the book is currently borrowed. This is the state the partial unique index checks against. It is not missing data — it is active state.
`loans`	`due_date`	**NO**	Every loan must have a due date at creation time. A loan without a due date is not a loan — it's chaos.
`fines`	`amount`	**NO**	A fine without an amount is meaningless. Combined with `CHECK (amount > 0)` this guarantees every fine row has a real, positive monetary value.
`payments`	`fine_id`	**NO**	A payment must always reference a specific fine. Orphaned payment records would break the audit trail and the fine-clearing workflow.
`media_items`	`file_path`	**YES**	Metadata-only records (e.g., catalog entries for physical-only items) may have no digital file. NULL here is valid and intentional.

Table 4.3 (continued)

Table	Column	Nullable?	Reasoning
`audit_log`	`actor_id`	**YES**	System-generated events (cron jobs, background workers) have no human actor. NULL actor = system action. A NOT NULL constraint here would prevent logging automated events.
`holds`	`notified_at`	**YES**	NULL means the hold is active but the item hasn't become available yet. Set to a timestamp when the notification is sent to the member.
`ai_chat_messages`	`model_used`	**NO**	Every AI response must record which model generated it. Essential for debugging quality issues, auditing, and understanding which Groq model tier handled which queries.

4.3.3 Check Constraints

Check constraints enforce business rules at the database level — the last line of defense when application code misbehaves.

Table 4.4: Check Constraints

Table	Constraint	What it prevents
`fines`	`CHECK (amount > 0)`	A fine of zero or negative is not a fine — it's a bug. Prevents the fine-worker from inserting a 0 BDT fine if it runs before the due date has actually passed.
`payments`	`CHECK (amount > 0)`	Prevents zero-value payment records, which would falsely mark a fine as paid without any real transaction.
`loans`	`CHECK (due_date > checkout_date)`	A loan due before it was checked out is a time-travel bug. Catches any clock-skew or misconfiguration in the loan period calculation.

Table 4.4 (continued)

Table	Constraint	What it prevents
media_items	CHECK (format IN (...))	Only supported formats allowed. Prevents a rogue upload endpoint from creating records with format 'exe' or 'unknown' that the media viewer can't handle.
vector_embeddings	CHECK (chunk_index >= 0)	Chunk indices are zero-based non-negative integers. A negative index indicates a bug in the text-splitting logic.
ai_chat_messages	CHECK (char_length(query) <= 5000)	Enforces the 5000-character query limit from the SRS data dictionary at DB level. Catches validation bugs in the API layer before they hit the LLM.
student_projects	CHECK (academic_year >= 2000 AND academic_year <= 2100)	Prevents data entry errors like 202 (missing digit) or 1995 (predates the dept going digital).

```sql
ALTER TABLE fines
    ADD CONSTRAINT chk_fine_positive CHECK (amount > 0);

ALTER TABLE payments
    ADD CONSTRAINT chk_payment_positive CHECK (amount > 0);

ALTER TABLE loans
    ADD CONSTRAINT chk_loan_dates CHECK (due_date >
       checkout_date);

ALTER TABLE media_items
    ADD CONSTRAINT chk_media_format
    CHECK (format IN ('pdf', 'video', 'image', 'audio', 'docx'));

ALTER TABLE vector_embeddings
    ADD CONSTRAINT chk_chunk_index CHECK (chunk_index >= 0);

ALTER TABLE ai_chat_messages
    ADD CONSTRAINT chk_query_length CHECK (char_length(query) <=
       5000);

ALTER TABLE student_projects
    ADD CONSTRAINT chk_academic_year
    CHECK (academic_year >= 2000 AND academic_year <= 2100);
```

Listing 4.2: Check constraints — DDL

4.3.4 Referential Integrity (Foreign Key Constraints)

Foreign keys are what turn a pile of tables into a relational database. Every FK constraint here is intentional — and so is every `ON DELETE` behaviour. The cascade strategy is the most consequential decision in each one.

Table 4.5: Foreign Key Constraints and Cascade Behaviour

Table	References	On Delete	Reasoning
`loans.user_id`	`users.user_id`	**RESTRICT**	Cannot delete a user with loan history. Delete the loans first, deliberately. Preserves the full borrowing audit trail.
`loans.item_id`	`media_items.item_id`	**RESTRICT**	Cannot delete an item that has ever been borrowed. Loan history is a permanent record.
`fines.loan_id`	`loans.loan_id`	**RESTRICT**	A fine record anchors to its originating loan. Deleting the loan would orphan the fine and destroy the financial audit trail.
`payments.fine_id`	`fines.fine_id`	**RESTRICT**	A payment cannot outlive the fine it paid. Orphaned payment records are financially meaningless and audit nightmares.
`vector_embeddings.item_id`	`media_items.item_id`	**CASCADE**	When a document is deleted, its embeddings go with it automatically. No reason to keep 50 orphaned vector chunks for a document that no longer exists.
`metadata.item_id`	`media_items.item_id`	**CASCADE**	Metadata has no meaning without its parent item. The only 1:1 relationship with cascade — all financial relationships are RESTRICT.

Table 4.5 (continued)

Table	References	On Delete	Reasoning
ai_chat_messages.user_id	users.user_id	**SET NULL**	If a user is deleted (GDPR request etc.), chat history becomes anonymised rather than deleted. The log is still useful for AI quality monitoring.
holds.user_id	users.user_id	**CASCADE**	If a user is deleted, their active holds are cancelled automatically. No one should inherit their queue position.
audit_log.actor_id	users.user_id	**SET NULL**	Audit records are permanent. If the user is deleted, the action stays but the actor becomes anonymous — satisfies both data retention and right-to-erasure simultaneously.

```sql
ALTER TABLE loans
    ADD CONSTRAINT fk_loans_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE RESTRICT,
    ADD CONSTRAINT fk_loans_item
        FOREIGN KEY (item_id) REFERENCES media_items(item_id)
        ON DELETE RESTRICT;

ALTER TABLE fines
    ADD CONSTRAINT fk_fines_loan
        FOREIGN KEY (loan_id) REFERENCES loans(loan_id)
        ON DELETE RESTRICT,
    ADD CONSTRAINT fk_fines_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE RESTRICT;

ALTER TABLE payments
    ADD CONSTRAINT fk_payments_fine
        FOREIGN KEY (fine_id) REFERENCES fines(fine_id)
        ON DELETE RESTRICT;

-- CASCADE: embeddings die with their document
ALTER TABLE vector_embeddings
    ADD CONSTRAINT fk_embeddings_item
        FOREIGN KEY (item_id) REFERENCES media_items(item_id)
```

```sql
        ON DELETE CASCADE;

-- CASCADE: metadata dies with its document
ALTER TABLE metadata
    ADD CONSTRAINT fk_metadata_item
        FOREIGN KEY (item_id) REFERENCES media_items(item_id)
        ON DELETE CASCADE;

-- SET NULL: anonymise chat history on user deletion
ALTER TABLE ai_chat_messages
    ADD CONSTRAINT fk_chat_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE SET NULL;

-- CASCADE: cancel holds when user is deleted
ALTER TABLE holds
    ADD CONSTRAINT fk_holds_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE CASCADE,
    ADD CONSTRAINT fk_holds_item
        FOREIGN KEY (item_id) REFERENCES media_items(item_id)
        ON DELETE RESTRICT;

-- SET NULL: preserve audit record, anonymise actor
ALTER TABLE audit_log
    ADD CONSTRAINT fk_audit_actor
        FOREIGN KEY (actor_id) REFERENCES users(user_id)
        ON DELETE SET NULL;
```

Listing 4.3: Foreign key constraints with cascade behaviour

4.3.5 Enum Constraints

PostgreSQL ENUMs are constraints that live at the type level — a column typed as an ENUM *cannot* hold an unlisted value. This is stronger than a CHECK constraint because it's enforced by the type system itself.

Table 4.6: PostgreSQL ENUM Types

ENUM Type	Allowed Values and Notes
`role_tier`	`'public'`, `'member'`, `'staff'`, `'admin'`, `'ai_admin'` — Ordered by privilege level. The RBAC middleware maps these to permission sets. Adding a new role requires a schema migration (`ALTER TYPE ... ADD VALUE`), which is intentional — it forces a deliberate decision.
`item_status`	`'draft'`, `'review'`, `'published'`, `'archived'` — The document lifecycle. State transitions enforced in the Go service layer. A document cannot jump from `'draft'` to `'published'` without passing through `'review'`.
`access_tier`	`'public'`, `'member'`, `'staff'`, `'restricted'` — Controls document visibility. The RAG service `WHERE` clause maps user `role_tier` to the set of `access_tier` values they can see.
`fine_status`	`'pending'`, `'paid'`, `'waived'` — `'waived'` allows a librarian to manually cancel a fine (e.g., system error). Distinct from `'paid'` — no payment record is created for a waived fine.
`payment_status`	`'pending'`, `'successful'`, `'failed'` — Reflects gateway response. Only `'successful'` clears the associated fine.
`hold_status`	`'active'`, `'fulfilled'`, `'cancelled'` — `'fulfilled'` = item became available and member was notified. `'cancelled'` = member withdrew the request manually.

```sql
CREATE TYPE role_tier AS ENUM (
    'public', 'member', 'staff', 'admin', 'ai_admin'
);
CREATE TYPE item_status AS ENUM (
    'draft', 'review', 'published', 'archived'
);
CREATE TYPE access_tier AS ENUM (
    'public', 'member', 'staff', 'restricted'
);
CREATE TYPE fine_status AS ENUM (
    'pending', 'paid', 'waived'
);
CREATE TYPE payment_status AS ENUM (
    'pending', 'successful', 'failed'
);
CREATE TYPE hold_status AS ENUM (
    'active', 'fulfilled', 'cancelled'
);

-- Adding a value later (non-destructive):
-- ALTER TYPE role_tier ADD VALUE 'external_researcher' AFTER
    'member';
-- Note: you cannot REMOVE a value from a PostgreSQL ENUM without
```

```
-- recreating the type entirely. Think before you add.
```

Listing 4.4: ENUM type definitions (run before CREATE TABLE)

4.3.6 Validation Constraints

Validation constraints sit at the *boundary* of the system — they are the first line of defence, firing before any data touches the database. They live in three places: the Next.js frontend (user experience), the Go API middleware (enforcement), and the RAG service (AI input sanitisation). A constraint that only exists in the frontend is not a constraint — it is a suggestion. Every rule here is enforced at the Go API layer at minimum, regardless of what the frontend does or doesn't do.

User Input Validation

Table 4.7: User Account Input Validation

Field	Rule	HTTP Error	Notes
`email`	Valid RFC 5322 format; not empty; max 254 chars	400 Bad Request	Validated with a proper regex, not `strings.Contains("@")`. 254 chars is the actual RFC maximum for an email address.
`password`	Min 8 chars; at least one uppercase, one lowercase, one digit	400 Bad Request	Validated before hashing. The rule lives in Go — the frontend shows hints but the API enforces it. Passwords over 72 chars are truncated silently by bcrypt, so we cap input at 128 chars too.
`name`	Not empty; min 2 chars; max 100 chars; no control characters	400 Bad Request	Strips leading/trailing whitespace. Rejects null bytes and ASCII control characters which can break downstream rendering.
`role_tier`	Must be a valid ENUM value	400 Bad Request	Only admins can set `role_tier` on registration. Self-registered users are always assigned `'member'` regardless of what they send.

File Upload Validation

Table 4.8: File Upload Validation Rules

Field	Rule	HTTP Error	Notes
`file format`	MIME type must match extension; must be in allowed list	415 Unsupported Media Type	Validated by reading the actual file magic bytes — not the `Content-Type` header, which the client can lie about. A file named `paper.pdf` that is actually a ZIP is rejected.
`file size`	PDF $\leq$ 100MB; Video $\leq$ 200MB; Image $\leq$ 10MB; Audio $\leq$ 50MB	413 Content Too Large	Checked from `Content-Length` header before streaming begins. If `Content-Length` is absent, the stream is read with a hard byte cap and rejected at the limit.
`title`	Not empty; min 3 chars; max 255 chars	400 Bad Request	Titles under 3 characters are almost certainly data entry errors. Max 255 matches the DB column width.
`abstract`	Max 5000 chars; UTF-8 encoded	400 Bad Request	Validated for valid UTF-8 encoding explicitly — malformed Bangla text (invalid byte sequences) is rejected with a descriptive error, not a cryptic 500.
`keywords`	Max 20 tags; each tag max 50 chars; no duplicates	400 Bad Request	Duplicate tags are silently deduplicated rather than rejected — this is a quality-of-life call. More than 20 tags is rejected because it suggests keyword stuffing rather than legitimate metadata.

Table 4.8 (continued)

Field	Rule	HTTP Error	Notes
`language`	Must be `'en'` or `'bn'`	400 Bad Request	ISO 639-1 codes only. Defaults to `'en'` if omitted. Bangla is `'bn'`, not `'bd'` — a common mistake.

Library Workflow Validation

Table 4.9: Library Operation Validation Rules

Field	Rule	HTTP Error	Notes
`barcode`	Alphanumeric only; min 4 chars; max 64 chars; not empty	400 Bad Request	Strips whitespace from scanner input — HID scanners sometimes append a trailing newline or carriage return. Validated before any DB lookup.
`loan_period`	Must be a positive integer; max 90 days	400 Bad Request	90 days is the absolute upper cap regardless of config. Prevents a misconfigured `.env` from creating loans due in the year 2087.
`fine amount`	Must be > 0; max 10,000 BDT per fine record	400 Bad Request	A single fine over 10,000 BDT almost certainly indicates a calculation bug, not a legitimate overdue penalty. The cap is configurable but exists.
`hold request`	User must be `'member'` or above; item must exist and be currently borrowed	403 / 404	You cannot place a hold on an available item — just borrow it. The API checks availability before creating the hold record.

`return date`	Cannot be before `checkout_date`; cannot be in the future	400 Bad Request	A return date in the future would mean the book is being returned before it is borrowed back in time, or the server clock is wrong. Both are problems.

AI Query Validation

Table 4.10: AI Query Input Validation Rules

Field	Rule	HTTP Error	Notes
`query`	Not empty; min 3 chars; max 5000 chars; valid UTF-8	400 Bad Request	Minimum 3 chars filters out accidental button mashes. Max 5000 chars mirrors the DB constraint and protects the Groq token budget. UTF-8 validation catches malformed Bangla before it reaches the embedding model.
`session_id`	Valid UUID v4 format if provided; auto-generated if absent	400 Bad Request	If the client sends a malformed session ID, the Go API rejects it rather than silently creating a broken session. A missing session ID is fine — the API generates one and returns it in the response.
`language`	`'en'`, `'bn'`, or `'auto'`	400 Bad Request	`'auto'` triggers `langdetect` in the RAG service. Explicit language selection bypasses detection and is faster. Defaults to `'auto'` if omitted.

Table 4.10 (continued)

Field	Rule	HTTP Error	Notes
Prompt injection	Query scanned for known injection patterns before forwarding to RAG service	400 Bad Request	Patterns like `"ignore previous instructions"` or `"you are now"` are flagged and rejected. Not foolproof, but catches the obvious attempts. More sophisticated jailbreak attempts are mitigated by the grounded RAG prompt design (model only answers from retrieved context).
`model_override`	Only `ai_admin` role may override the model tier selection	403 Forbidden	Regular users cannot force the system to use `gpt-oss-120b` for every query and burn through the daily quota. Model routing is server-side only.

Metadata and Research Submission Validation

Table 4.11: Research and Project Submission Validation

Field	Rule	HTTP Error	Notes
`authors`	At least one author required; max 20 authors; each author must be a valid `user_id` or a free-text name string	400 Bad Request	A research paper with zero authors cannot be submitted. Free-text author names (for external co-authors not in the system) are allowed but capped at 100 chars each.
`academic_year`	Integer; ≥ 2000; $\leq$ current year + 1	400 Bad Request	Current year + 1 allows for projects submitted slightly ahead of the academic calendar. Anything beyond that is a data entry error.

Table 4.11 (continued)

Field	Rule	HTTP Error	Notes
`supervisor`	Must be a valid `user_id` with `role_tier` ≥ `'staff'`	400 Bad Request	A student cannot supervise another student's project. The supervisor field is validated against the users table at submission time.
`team_members`	At least one member; max 10 members; all must be valid `user_id` values	400 Bad Request	A project with no team members is orphaned. A project with 11 members is almost certainly a data entry mistake or someone adding their entire friend group for clout.
`doi`	Optional; if provided, must match DOI format `10.XXXX/...`	400 Bad Request	DOI validation uses a standard regex. An invalid DOI silently breaks citation linking later — better to reject it at entry time.

Validation Implementation in Go

All validation in the Go API layer uses a single consistent approach — struct tags with a validation library, applied in middleware before any handler logic runs. This keeps validation out of business logic entirely.

```go
// internal/models/upload.go

type DocumentUploadRequest struct {
    Title    string   `json:"title"
                      validate:"required,min=3,max=255"`
    Abstract string   `json:"abstract"
                      validate:"max=5000"`
    Keywords []string `json:"keywords"
                      validate:"max=20,dive,max=50"`
    Language string   `json:"language"
                      validate:"oneof=en bn"`
    Format   string   `json:"format"
                      validate:"required,oneof=pdf video image
                         audio docx"`
}

type AIQueryRequest struct {
    Query     string `json:"query"
                     validate:"required,min=3,max=5000"`
```

```go
    SessionID string `json:"session_id"
                      validate:"omitempty,uuid4"`
    Language  string `json:"language"
                      validate:"omitempty,oneof=en bn auto"`
}

type LoanCreateRequest struct {
    MemberBarcode string `json:"member_barcode"
                          validate:"required,min=4,max=64,alphanum"
    ItemBarcode   string `json:"item_barcode"
                          validate:"required,min=4,max=64,alphanum"
}

// Validation middleware     runs before every handler
func ValidateRequest[T any](c *gin.Context) (T, error) {
    var req T
    if err := c.ShouldBindJSON(&req); err != nil {
        return req, fmt.Errorf("malformed JSON: %w", err)
    }
    validate := validator.New()
    if err := validate.Struct(req); err != nil {
        return req, fmt.Errorf("validation failed: %w", err)
    }
    return req, nil
}
```

Listing 4.5: Go validation struct example (using go-playground/validator)

4.3.7 Application-Level Constraints

Some constraints are too complex for SQL — they require business logic in the Go service layer. These are still real constraints; they just live higher up the stack. Document them here because a new developer writing a new endpoint can break them just as easily as a missing DB constraint.

Table 4.12: Application-Level Constraints (Enforced in Go Service Layer)

Rule	Description
Document state machine	`draft` → `review` → `published` → `archived` only. No skipping, no reverting (except admin override). Enforced in `internal/archive/state.go`.
Checkout block on fines	A member with unpaid fines > 50 BDT cannot borrow. Checked before any DB write. Threshold is configurable via `FINE_BLOCK_THRESHOLD_BDT` in `.env`.
File size limits	PDF ≤ 100MB, Video ≤ 200MB, Image ≤ 10MB, Audio ≤ 50MB. Enforced in upload middleware before the file reaches MinIO. HTTP 413 on violation.
Loan period calculation	`due_date = checkout_date + LOAN_PERIOD_DAYS` (default 14). Calculated in Go so the period always reflects the current config value, not a hardcoded DB default.
AI query model routing	The RAG service selects the Groq model tier (8B / 17B / 120B) based on query complexity heuristics. Simple queries always use the 8B model to preserve the 120B daily quota for complex reasoning.
Session context window	AI multi-turn chat passes at most the last 10 messages per `session_id`. Hard cap to prevent context window overflow and excessive Groq token usage (FR-AI-008).
Reviewer cannot self-approve	A researcher who submitted a document cannot be its own reviewer. Enforced in the review assignment handler before the reviewer is written to the DB.

4.4 Critical Schema Snippets

```sql
-- Enable pgvector extension (run once)
CREATE EXTENSION IF NOT EXISTS vector;

-- Enums (define before tables)
CREATE TYPE role_tier AS ENUM
   ('public','member','staff','admin','ai_admin');
CREATE TYPE item_status AS ENUM
   ('draft','review','published','archived');
CREATE TYPE access_tier AS ENUM
   ('public','member','staff','restricted');
CREATE TYPE fine_status AS ENUM ('pending','paid','waived');
```

```sql
CREATE TYPE payment_status AS ENUM
    ('pending','successful','failed');

-- Users
CREATE TABLE users (
    user_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email        TEXT NOT NULL UNIQUE,
    name         TEXT NOT NULL,
    password_hash TEXT,                -- NULL for SSO-only users
    role_tier    role_tier NOT NULL DEFAULT 'member',
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    last_login   TIMESTAMPTZ
);

-- Media items (archive + research + projects)
CREATE TABLE media_items (
    item_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title        TEXT NOT NULL,
    item_type    TEXT NOT NULL,        --
        'archive','research','project'
    format       TEXT NOT NULL,        --
        'pdf','video','image','audio'
    status       item_status NOT NULL DEFAULT 'draft',
    access_tier  access_tier NOT NULL DEFAULT 'public',
    created_by   UUID REFERENCES users(user_id),
    file_path    TEXT,                 -- MinIO object key
    upload_date  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Loans      the partial unique index is the hero here
CREATE TABLE loans (
    loan_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id        UUID NOT NULL REFERENCES users(user_id),
    item_id        UUID NOT NULL REFERENCES media_items(item_id),
    checkout_date  TIMESTAMPTZ NOT NULL DEFAULT now(),
    due_date       TIMESTAMPTZ NOT NULL,
    return_date    TIMESTAMPTZ          -- NULL = currently
        borrowed
);

-- This single index prevents double-borrowing without
    application logic
CREATE UNIQUE INDEX idx_loans_active_item
    ON loans(item_id)
    WHERE return_date IS NULL;

-- Fines      NUMERIC not FLOAT. Never use FLOAT for money.
CREATE TABLE fines (
    fine_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    loan_id        UUID NOT NULL REFERENCES loans(loan_id),
    user_id        UUID NOT NULL REFERENCES users(user_id),
```

```sql
    amount          NUMERIC(10,2) NOT NULL CHECK (amount > 0),
    status          fine_status NOT NULL DEFAULT 'pending',
    calculated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Vector embeddings for RAG
CREATE TABLE vector_embeddings (
    embedding_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    item_id       UUID NOT NULL REFERENCES media_items(item_id)
       ON DELETE CASCADE,
    chunk_index   INT NOT NULL,
    chunk_text    TEXT NOT NULL,
    embedding     vector(768) NOT NULL  --
       paraphrase-multilingual-MiniLM-L12-v2
);

-- HNSW index for fast approximate nearest neighbour search
-- m=16, ef_construction=64 are good defaults for our scale
CREATE INDEX idx_embeddings_hnsw
    ON vector_embeddings
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- Audit log     append only, no updates ever
CREATE TABLE audit_log (
    log_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id      UUID REFERENCES users(user_id),
    action        TEXT NOT NULL,
    resource_type TEXT NOT NULL,
    resource_id   UUID,
    ip_addr       INET,
    timestamp     TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Prevent updates and deletes via row-level security
ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;
CREATE POLICY audit_insert_only ON audit_log FOR INSERT WITH
   CHECK (true);
```

Listing 4.6: PostgreSQL schema — critical tables

4.5 Data Lifecycle Notes

- **Documents:** `draft` → `review` → `published` → `archived`. State transitions are enforced in the Go service layer. No document can skip the review step to become published.
- **Embeddings:** Regenerated every time document content or metadata changes. The ingestion worker handles this asynchronously — the document is immediately available in keyword search; semantic search becomes available after the embedding job completes.

- **Fines:** Calculated nightly by the fine-worker cron. Rate is configurable per admin (default 50 BDT/day). Maximum fine cap is configurable (decided by library administration per SRS open question #2).
- **Audit logs:** Retained for 90 days in PostgreSQL hot storage. Older records archived to cold storage (MinIO or filesystem). No record is ever deleted.
- **AI chat history:** Retained per user session. Used as context window for multi-turn conversations (up to 10 previous messages per `session_id` as per FR-AI-008).

Chapter 5

Core Flow Design

5.1 Flow 1: Document Upload → RAG Ingestion Pipeline

This is the most technically complex flow and the backbone of the entire AI system. If this breaks, the AI has nothing to search.

Trigger: A staff member or researcher uploads a file via the frontend.

1. Next.js sends a `multipart/form-data` POST to `Go API: POST /api/v1/documents`
2. Go API validates: file type (PDF/video/image/audio), file size (PDF $\leq$ 100MB, video $\leq$ 200MB), and user's role tier ($\geq$ `staff`)
3. File binary streamed to MinIO; metadata written to `media_items` and `metadata` tables with `status = 'draft'`
4. Go API pushes a job to Redis: `{"job": "ingest", "item_id": "...", "file_path": "..."}`
5. HTTP 201 returned to frontend immediately — the user doesn't wait for ingestion
6. **Ingestion Worker** (Go) picks up the Redis job
7. Text extracted from the file using `PyMuPDF` (called via Python subprocess or RPC)
8. Text split into overlapping chunks (512 tokens, 50-token overlap) to preserve context across chunk boundaries
9. Each chunk sent to `paraphrase-multilingual-MiniLM-L12-v2` (runs locally in Docker) — returns 768-dim vector
10. Vectors + chunk text written to `vector_embeddings` table via pgvector
11. `media_items.status` updated; full-text search index refreshed
12. **Success:** Document is searchable via keyword search immediately after step 3; semantic search available after step 10

Failure and edge cases:

- **Corrupt PDF:** PyMuPDF raises exception → job marked `ingestion_failed` in DB → uploader notified via dashboard alert. Document remains accessible as a binary download; semantic search just won't work for it.
- **Groq API down during ingestion:** Doesn't matter — embeddings are generated locally. LLM is only called at query time, not ingestion time. This is an intentional architectural decision.
- **File exceeds size limit:** Rejected at Go API layer (step 2) before any storage is touched. HTTP 413 returned with a friendly message.
- **Redis queue down:** Go API falls back to synchronous ingestion inline (slower, but document still gets indexed). Logged as an alert to monitoring.

5.2 Flow 2: RAG AI Query with Access Control

This is the flow that makes the AI actually trustworthy — access control is enforced at the retrieval layer, not as an afterthought on the response.

Trigger: User submits a natural language query in the AI chat widget.

1. Next.js sends `POST /api/v1/ai/query` with `{query, session_id, language}`
2. Go API extracts JWT, validates it, and determines `user_role` and `accessible_tiers`
3. Request forwarded to RAG service with role context: `{query, role_tier, session_id}`
4. RAG service detects query language (English/Bangla) via `langdetect`
5. **(Optional) Query rewriting:** Gemini refines ambiguous or poorly structured queries for better retrieval
6. Query embedded using `paraphrase-multilingual-MiniLM-L12-v2` locally
7. **Hybrid retrieval executed:**

```sql
WITH vector_results AS (
  SELECT ve.chunk_text, ve.item_id,
         1 - (ve.embedding <=> $1) AS score
  FROM vector_embeddings ve
  JOIN media_items mi ON ve.item_id = mi.item_id
  WHERE mi.access_tier = ANY($2)
    AND mi.status = 'published'
  ORDER BY ve.embedding <=> $1
  LIMIT 8
),
fts_results AS (
  SELECT ve.chunk_text, ve.item_id,
         ts_rank_cd(to_tsvector('english', ve.chunk_text),
                    plainto_tsquery($3)) AS score
  FROM vector_embeddings ve
  JOIN media_items mi ON ve.item_id = mi.item_id
  WHERE mi.access_tier = ANY($2)
```

```
    AND mi.status = 'published'
    AND to_tsvector('english', ve.chunk_text) @@
        plainto_tsquery($3)
  ORDER BY score DESC
  LIMIT 8
)
SELECT * FROM vector_results
UNION
SELECT * FROM fts_results
LIMIT 10;
```

Listing 5.1: Hybrid RAG query with access control (vector + FTS)

8. Top-K chunks retrieved, merged, and ranked
9. Prompt constructed:

```
system_prompt = """You are the CSEDU Knowledge Assistant.
Answer ONLY using the provided context.
For every factual claim, cite the document ID in [brackets].
If the context doesn't contain the answer, say so honestly.
Never hallucinate sources. Never make up facts."""

user_prompt = f"""Context:
{chr(10).join([f'[{c["item_id"]}] {c["chunk_text"]}' for c
    in chunks])}

Question: {query}
Answer in: {detected_language}"""
```

Listing 5.2: RAG prompt construction

10. **Primary LLM call:** Groq API is called with the appropriate model tier
11. **Fallback logic:** If Groq fails (rate limit, timeout, or error), request is automatically routed to Gemini 2.5 Flash
12. Response + citation list stored in `ai_chat_messages`; returned to frontend

Success outcome: Grounded, cited response in the user's language within 10 seconds.

Failure and edge cases:

- **No relevant documents found:** Return *"I couldn't find relevant information in the platform's documents."* — never hallucinate an answer
- **Groq rate limit hit (HTTP 429):** Exponential backoff (1s, 2s, 4s), then automatic fallback to Gemini
- **Restricted document attempted:** The retrieval `WHERE` clause filters it out before it reaches the LLM — restricted data never enters the prompt
- **Groq service down:** Immediate fallback to Gemini ensures uninterrupted AI service

- **Gemini also unavailable:** Fallback to keyword-only (FTS) results with explanation that AI generation is temporarily degraded
- **Malformed or vague query:** Query rewriting via Gemini improves retrieval quality before embedding

5.3 Flow 3: Borrow a Library Item (Barcode Workflow)

Trigger: Librarian scans a member card and a book barcode at the circulation desk.

1. Librarian's HID-compatible scanner inputs barcode string into the browser input field
2. Next.js sends `POST /api/v1/loans` with `{member_barcode, item_barcode}`
3. Go API looks up user by `member_barcode`; looks up item by `item_barcode`
4. Validation checks in order:
   a. Is the item currently borrowed? (checks partial unique index — fast)
   b. Does the member have unpaid fines exceeding 50 BDT? (FR-TXX-018: checkout block)
   c. Is the member's account active?
5. If all checks pass: insert `loans` record with `due_date = now() + library_loan_period`
6. Update item status; transaction committed atomically
7. Return loan details to frontend (loan ID, due date, member name)

Failure and edge cases:

- **Item already borrowed:** HTTP 409 Conflict — "This item is currently on loan to another member." Hold request offered.
- **Unpaid fines block:** HTTP 403 — "Member has outstanding fines of X BDT. Payment required before borrowing."
- **Invalid barcode:** HTTP 404 — retry option presented immediately
- **Network failure mid-transaction:** PostgreSQL transaction rolls back — no partial state, no ghost loans (NFR-TXX-024: ACID compliance)

5.4 Flow 4: SSO Login (OAuth 2.0)

Trigger: User clicks "Login with Institution" on the login page.

1. Next.js redirects user to `GET /api/v1/auth/sso/login`
2. Go API constructs the OAuth 2.0 authorization URL with `state` parameter (CSRF protection) and redirects to IdP
3. User authenticates at institutional identity provider

4. IdP redirects to `GET /api/v1/auth/sso/callback?code=...`
5. Go API exchanges authorization code for access token via back-channel POST to IdP
6. User profile fetched from IdP; matched to existing `users` record by email, or new record created
7. JWT access token (1hr) + refresh token (7 days) issued; stored in HTTP-only cookies
8. User redirected to dashboard with appropriate role

Failure and edge cases:

- **SSO provider unavailable:** Fall back to local email/password login (local auth is always maintained as fallback per design decision)
- **State mismatch:** CSRF attack detected — session terminated, HTTP 403 returned
- **5 failed login attempts:** Account locked for 15 minutes (FR-TXX-007)

5.5 Flow 5: Research Paper Submission with Review Workflow

Trigger: A researcher submits a paper draft via the research portal.

1. Researcher uploads PDF + metadata (title, abstract, co-authors, keywords) via frontend
2. Go API validates and stores document with `status = 'draft'` — invisible to public
3. AI ingestion pipeline runs asynchronously (Flow 1) — AI will be able to summarize it once indexed
4. Researcher can request review: `PATCH /api/v1/research/:id/submit-for-review`
5. Document status → `review'`; reviewer (staff) assigned and notified
6. Reviewer reads, optionally uses AI summary to accelerate review, then approves or rejects
7. On approval: status → `published`; document becomes publicly searchable
8. AI auto-generates keywords and subject classifications (FR-AI-012); researcher can edit

Failure and edge cases:

- **Reviewer rejects:** Status reverts to `draft`; rejection reason stored; researcher notified with comments
- **Metadata incomplete:** Validation at submission time — required fields (title, abstract, at least one author) enforced before status changes

Chapter 6

Security and Access Design

6.1 Authentication Approach

- **Primary:** OAuth 2.0 / SAML SSO via institutional identity provider
- **Fallback:** Local email + password authentication (always maintained — SSO providers go down)
- **Token mechanism:** JWT access tokens (1-hour expiry) + refresh tokens (7-day expiry) stored in HTTP-only cookies (not localStorage — XSS protection)
- **Password hashing:** bcrypt with cost factor 12 (never MD5, never SHA1, never plain text — this is non-negotiable)
- **Session invalidation:** Refresh tokens stored in `user_sessions` table; server-side invalidation on logout
- **Brute force protection:** Account locked after 5 failed login attempts per minute (FR-TXX-007, NFR-TXX-039)

6.2 Authorization Model (RBAC)

Table 6.1: Role-Based Access Control Matrix

Permission	Public	Member	Staff	Admin	AI Admin
View public content					
View member content		×			
View staff content			×	×	
View restricted content			×	×	×
Borrow / hold books		×		×	
Upload documents			×	×	×
Approve / publish			×	×	×
Manage users + roles			×	×	×
AI chat (basic)					
AI chat (personalized)		×			
Configure AI models			×	×	×
View AI performance			×	×	×

RBAC is enforced in Go middleware on **every** request. There is no client-side enforcement — the frontend hiding a button is not security.

6.3 Sensitive Data Handling

- **Passwords:** bcrypt (cost 12) — stored hash only, never recoverable
- **All traffic:** HTTPS/TLS enforced at Nginx layer — no plain HTTP in production
- **Stored files:** MinIO server-side encryption enabled for all uploaded content
- **Payment data:** No raw card data ever stored — gateway tokenization only (when gateway is integrated post-MVP)
- **PII (names, emails):** Stored in PostgreSQL; access restricted to authenticated sessions; never logged in plain text
- **JWT tokens:** HTTP-only cookies; `SameSite=Strict`; never accessible via JavaScript
- **API keys (Groq):** Stored only in `.env` files (Docker secrets in prod); never committed to git (enforce with `.gitignore` + pre-commit hook)

6.4 AI-Specific Security

This is the most important security section for this project — the AI access control must be enforced at the **retrieval layer**, not post-generation.

> **The golden rule:** Restricted documents must be filtered in the pgvector `WHERE` clause *before* any chunk is returned. They must never enter the LLM prompt. If a restricted document enters the prompt, the LLM might reference it even if instructed not to. Defense in depth: filter it out entirely so it never has the chance.

- Access tier filter injected into every pgvector query (see Flow 2, step 6)
- RAG service receives user's role tier from Go API on every request — it never trusts the client
- Restricted content queries return HTTP 403 with *"Access Denied"* before RAG query is even executed (FR-AI-018)
- AI responses include source citations — every factual claim is traceable and verifiable (FR-AI-005, NFR-TXX-019)
- AI-generated outputs flagged as supplementary — never authoritative source replacements

6.5 Audit and Logging Plan

- Every CRUD operation, login attempt, access denial, and AI query logged to `audit_log` table
- Logging handled transparently by Go middleware — individual service handlers don't call it manually
- Log retention: $\geq$ 90 days in PostgreSQL (NFR-TXX-017); archived to MinIO beyond that
- Logs include: actor, action, resource type, resource ID, IP address, timestamp
- `audit_log` is append-only via PostgreSQL row-level security — even admins cannot delete audit records
- Prometheus metrics exported from Go API and RAG service; Grafana dashboards for monitoring

Chapter 7

Implementation Plan

7.1 Sprint 1 — Completed Features (Core Foundation)

- User registration and local JWT authentication
- Basic role-tier assignment (Public / Member / Staff / Admin)
- Media upload endpoint with file size validation and MinIO storage
- PostgreSQL schema migrated with all core tables
- Basic library catalog browse and keyword search
- Docker Compose setup with all services running locally

7.2 Sprint 2 — Pending Features (Full Platform + AI MVP)

- Loan and return workflow (core happy path)
- Fine calculation logic (basic, manual trigger)
- Full RBAC enforcement across all Go API endpoints (middleware layer)
- Document ingestion pipeline: Redis queue → text extraction → pgvector embeddings
- RAG query engine with Groq integration and citation support
- AI chat widget (Next.js floating widget, multi-turn context)
- Tiered Groq model selection (8B/17B/120B based on query complexity)
- Research repository submission + review workflow
- Student project showcase with staff approval flow
- Fine worker cron (nightly automatic calculation)
- Fine payment flow (internal tracking; gateway stubbed)

- Barcode/QR scanner input for librarian checkout/return
- Librarian operations dashboard (overdue alerts, bulk import)
- SSO OAuth 2.0 integration (or stub with fallback to local auth)
- Bilingual support (English + Bangla) across UI and search
- Document summarization endpoint (FR-AI-003)
- Content classification and auto-tagging (FR-AI-012)
- Member dashboard (active loans, history, fines, wishlist)

7.3 Technical Risks and Mitigation

Table 7.1: Technical Risk Register

ID	Risk	Prob.	Impact	Mitigation
R-01	AI hallucinations in responses	Medium	High	Mandatory citation enforcement in RAG prompt; pgvector retrieval always required (no pure generation); user feedback loop
R-02	pgvector performance at 100K+ documents	Low	High	HNSW index (m=16, ef=64); load test before launch; fallback to keyword search if ANN latency exceeds 5s
R-03	Groq free tier rate limits hit during demo	Medium	Medium	Tiered model selection (cheap 8B for simple queries); response caching in Redis for repeated queries; graceful fallback message
R-04	SSO integration delayed	Medium	Medium	Build local JWT auth first (already done in Sprint 1); SSO is a drop-in replacement via OAuth abstraction layer
R-05	Bangla NLP quality poor	Low	Medium	`paraphrase-multilingual-MiniLM-L12-v2` tested on Bangla; native speaker review of outputs; separate BN tokenization

Table 7.1 (continued)

ID	Risk	Prob.	Impact	Mitigation
R-06	Legacy data migration corrupt	Medium	High	Dry-run migrations on staging DB; validate all required metadata fields; incremental migration; maintain rollback backup
R-07	Docker Compose resource limits on lab server	Low	Medium	Profile memory usage of all containers; set resource limits in compose file; MinIO and pgvector are the heaviest consumers
R-08	LLM context window exceeded for long PDFs	Low	Medium	Chunk size 512 tokens with overlap ensures no single chunk exceeds context; llama-4-scout-17b has 30K TPM headroom

Chapter 8

Testing Plan

We test because we're professionals, not because we're told to. Here's what we're targeting and — importantly — what's genuinely hard to test and why.

8.1 Unit Testing Targets

Table 8.1: Unit Test Targets

Component	Framework	What we test
Go API	`go test`	Fine calculation logic, JWT validation, RBAC permission checks, barcode parsing, loan due date calculation
Next.js	Jest + RTL	UI component rendering, form validation, AI chat widget state management
RAG Service	pytest	Prompt construction, citation extraction from LLM response, access tier filtering, language detection
Ingestion Worker	`go test`	Text chunking logic (512-token splits with overlap), embedding dimension validation

Target: **80%+ code coverage** on business logic. We are not going to write tests for Go's standard library.

8.2 Integration Testing Targets

- Full borrow/return/fine cycle: checkout → overdue → fine calculation → payment → restriction lift
- Document upload → MinIO storage → Redis job → ingestion worker (chunking + MiniLM embedding) → pgvector storage → hybrid retrieval returns indexed content
- SSO token exchange: mock OAuth callback → JWT issuance → authenticated request

- AI query pipeline (full RAG): user query → (optional Gemini rewrite) → MiniLM embedding → hybrid retrieval (pgvector + FTS with access filter) → Groq call → citation-grounded response
- **LLM fallback validation:** simulate Groq failure (HTTP 429/timeout) → automatic fallback to Gemini → response returned successfully
- **Access control enforcement:** restricted documents present in DB → query issued by low-tier user → verify exclusion at retrieval layer (not post-response)
- **Embedding consistency test:** ensure ingestion and query embeddings use same MiniLM model → validate similarity search correctness
- **Hybrid retrieval correctness:** verify vector-only vs FTS-only vs combined results → ensure relevant documents are not missed
- **Cache behavior (Redis):** repeated query → cache hit → reduced response latency
- Framework: Postman collections for Go API; pytest with test database for RAG service

8.3 End-to-End Testing Targets

- **Student borrowing journey:** Register → login → search catalog → borrow book → view dashboard → return → check fine balance
- **Librarian checkout:** Login → open librarian dashboard → scan member barcode → scan book barcode → confirm loan → verify item status changes
- **Researcher upload and discoverability:** Login → upload PDF → enter metadata → submit for review → staff approves → ingestion pipeline completes → document searchable via AI
- **AI query full pipeline (normal case):** user query → query rewrite (optional) → hybrid retrieval → Groq response → citations displayed in UI
- **AI fallback scenario:** simulate Groq failure → Gemini handles request → user still receives response without interruption
- **AI access control:** login as restricted user → query AI about protected document → verify no leakage of restricted content
- **AI degraded mode:** disable both Groq and Gemini → fallback to keyword-only (FTS) results → UI shows graceful degradation message
- **Multilingual query handling:** Bangla query → language detection → retrieval → response generated in Bangla
- **Session continuity:** multiple queries in same session → stored in `ai_chat_messages` → history correctly rendered
- **Performance target validation:** typical query → full pipeline execution → response returned within 10 seconds
- Framework: Cypress for browser-based E2E; test environment runs full Docker Compose stack

8.4 Known Hard-to-Test Areas

These are the things that are genuinely difficult to automate — be honest about them.

Table 8.2: Hard-to-Test Areas

Area	Why it's hard + what we do instead
LLM response quality	Non-deterministic. Can't assert exact string output. Mitigation: human evaluation with a fixed benchmark query set; measure citation presence and coherence score.
Bangla language accuracy	Requires native speaker evaluation. Mitigation: manual test suite of 20 Bangla queries reviewed by team members or volunteers from dept.
SSO integration	Real institutional IdP may not be available in CI. Mitigation: mock OAuth server in test environment; test against real IdP manually before release.
Groq rate limit behaviour	Can't trigger 429 in unit tests without burning real quota. Mitigation: mock the Groq client in tests; manually test the retry + fallback path once with real API.
Payment gateway callbacks	Webhook delivery is environment-dependent. Mitigation: use Stripe test mode with local webhook forwarding (ngrok or Stripe CLI) for manual testing.
Barcode scanner hardware	HID input device not available in CI. Mitigation: scanner input field accepts keyboard input as fallback — testable via Cypress type() events.

If a component is hard to unit test, that's a design smell. Tight coupling is the most common reason. In our case, the RAG service is designed as a pure function: inputs are (query, role_tier, session_history), output is (response, citations). It has no side effects except writing to the chat log. This design makes it unit-testable with mocked pgvector and mocked Groq client — no real API calls needed.

Chapter 9

Docker Deployment Design

9.1 Service Inventory

All services run inside Docker Compose. No external cloud dependencies for MVP (that's the entire point of our free stack).

Table 9.1: Docker Services

Service	Image / Build	Port(s)	Purpose
`frontend`	Build: ./frontend	3000	Next.js UI with SSR
`api`	Build: ./api	8080	Go monolith API server
`rag`	Build: ./rag	8001	Python FastAPI RAG service
`ingestion-worker`	Build: ./api (Dockerfile.worker)	—	Async document ingestion
`fine-worker`	Build: ./api (Dockerfile.fineworker)	—	Nightly fine cron
`postgres`	pgvector/pgvector:pg15	5432	DB + vector search
`redis`	redis:7-alpine	6379	Job queue + cache
`minio`	minio/minio:latest	9000, 9001	File storage
`nginx`	nginx:alpine	80, 443	Reverse proxy
`prometheus`	prom/prometheus (dev profile)	9090	Metrics collection
`grafana`	grafana/grafana (dev profile)	3001	Metrics dashboards

9.2 Dockerfile Strategy

Every production Dockerfile uses multi-stage builds. This keeps image sizes small and attack surfaces minimal.

- **Next.js:** Stage 1 = Node.js 20 builder (`npm run build`). Stage 2 = Node.js 20 Alpine runner with only the `.next/standalone` output. Final image ≈ 200MB.

- **Go API:** Stage 1 = golang:1.22-alpine (compile to static binary with `CGO_ENABLED=0`). Stage 2 = `gcr.io/distroless/static-debian12` — minimal attack surface, no shell, no package manager. Final image ≈ 20MB.
- **Workers:** Same Dockerfile as Go API, different `CMD` pointing to the worker binary entrypoint. No code duplication.
- **RAG Service:** python:3.11-slim with `sentence-transformers` + `PyMuPDF` + `fastapi` + `uvicorn`. The sentence-transformers model is downloaded at build time and baked into the image so cold start is fast.

9.3 Nginx Routing

```
upstream frontend { server frontend:3000; }
upstream api      { server api:8080; }
upstream rag      { server rag:8001; }

server {
    listen 80;

    # Static assets     served directly from Next.js
    location /_next/static/ { proxy_pass http://frontend; }

    # All API traffic
    location /api/ {
        proxy_pass http://api;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header Authorization $http_authorization;
    }

    # AI queries (larger timeouts     Groq can take up to 10s)
    location /api/v1/ai/ {
        proxy_pass http://api;
        proxy_read_timeout 30s;
        proxy_connect_timeout 5s;
    }

    # Frontend     catch-all
    location / {
        proxy_pass http://frontend;
        proxy_set_header Host $host;
    }
}
```

Listing 9.1: nginx.conf — path-based routing

9.4 Environment Variables

All secrets live in `.env` — never committed to git. A `.env.example` is committed with placeholder values.

```
# Database
DB_USER=csedu_user
DB_PASSWORD=changeme_in_prod
DB_NAME=csedu_platform

# Redis
REDIS_URL=redis://redis:6379

# MinIO
MINIO_USER=minioadmin
MINIO_PASSWORD=changeme_in_prod
MINIO_BUCKET=csedu-files

# JWT
JWT_SECRET=super_secret_jwt_key_change_this
JWT_EXPIRY_HOURS=1
REFRESH_EXPIRY_DAYS=7

# Groq (free tier     no credit card needed)
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxx
GROQ_MODEL_SIMPLE=llama-3.1-8b-instant
GROQ_MODEL_LONG=meta-llama/llama-4-scout-17b-16e-instruct
GROQ_MODEL_COMPLEX=openai/gpt-oss-120b

# SSO (fill when IT provides credentials)
SSO_CLIENT_ID=
SSO_CLIENT_SECRET=
SSO_REDIRECT_URI=http://localhost/api/v1/auth/sso/callback
SSO_PROVIDER_URL=

# Fine calculation
FINE_RATE_BDT_PER_DAY=50
FINE_BLOCK_THRESHOLD_BDT=50
LOAN_PERIOD_DAYS=14
```

Listing 9.2: .env.example

Chapter 10

Exit Checklist

- System overview is complete — problem, users, goals, and out-of-scope are all documented
- Architecture and component breakdown are documented with responsibilities and data flows
- All key design decisions are documented with options considered and reasons chosen
- Data model is documented with entities, relationships, constraints, and lifecycle notes
- At least 2 critical flows are documented
- Security and access model is defined — auth, RBAC, data handling, AI access control
- Sprint 1 and Sprint 2 features are listed with technical risks and mitigations
- Testing plan covers unit, integration, E2E levels plus honest hard-to-test areas
- Docker deployment design covers all services, Dockerfiles, routing, and secrets
- All open questions from SRS v2 Section 11.2 have been answered in the relevant sections
- Repository URL and commit hash are filled in before final submission
- `.env.example` is committed to repo (with placeholder values, not real secrets)
- This document has been reviewed by at least one team member who didn't write it

— Team Devops, 14th April 2026

Appendix A

SRS Open Questions — Resolution Status

The SRS v2 Section 11.2 listed 10 open questions. Here is how this SDD addresses each:

Table A.1: SRS Open Questions Resolution

#	Question	Decision in SDD
1	Cloud storage provider	**MinIO** (self-hosted, S3-compatible). Zero cost. Config-swap to real S3 post-MVP.
2	Fine calculation rules	Default 50 BDT/day, block at 50 BDT outstanding. Both configurable via `.env`. Library admin sets final values.
3	LLM provider and model	**Groq (primary) + Gemini 2.5 Flash (fallback)**. Groq handles main inference (tiered models for simple/complex queries). Gemini used for fallback, query rewriting, and summarization.
4	Vector database	**pgvector** (PostgreSQL extension) with HNSW index + PostgreSQL Full-Text Search (FTS) for hybrid retrieval. No separate infrastructure.
5	Authentication / SSO	**OAuth 2.0** via institutional IdP. Local JWT as always-maintained fallback.
6	Budget constraints	**Zero external API cost (primary path)**. Groq free tier + local MiniLM embeddings + self-hosted MinIO. Gemini free tier used only for fallback and auxiliary tasks. Server cost only.

Table A.1 (continued)

#	Question	Decision in SDD
7	Timeline confirmation	MVP targets Sprint 2 completion. Specific date TBD with supervisor.
8	Bangla content volume	Unknown. System supports multilingual queries via MiniLM embeddings and language detection. Separate PostgreSQL FTS configuration for Bangla content.
9	Data governance	Audit logs retained 90 days in DB, archived to MinIO beyond. Research papers follow document lifecycle. No auto-deletion.
10	AI governance	AI responses are supplementary only; all include citations (NFR-TXX-019). Low-confidence queries trigger fallback. No hallucination policy enforced at prompt level. No separate AI ethics committee for MVP scope.
11	Embedding strategy	**MiniLM (sentence-transformers, local)** used exclusively for both ingestion and query embedding. Ensures consistent vector space and avoids cross-model incompatibility.
12	Retrieval strategy	**Hybrid retrieval (vector + keyword)** combining pgvector similarity search and PostgreSQL FTS. Improves recall and precision over vector-only search.

Appendix B

Groq Model Reference

Table B.1: Groq Free Tier Models Used in This Project

Model	RPM	RPD	Context	Use in this project
`llama-3.1-8b-instant`	30	14,400	128K	Simple catalog queries, availability checks, basic chat
`llama-4-scout-17b`	30	1,000	128K	Long PDF summarization (up to 5 pages), large context tasks
`gpt-oss-120b`	30	1,000	128K	Complex reasoning, research insights, multi-turn sessions

GPT-OSS 120B is OpenAI's open-weight MoE model with 120B total parameters (5.1B active per token). It achieves near-parity with o4-mini on reasoning benchmarks and runs at 500+ t/s on Groq's LPU hardware. It is available on the Groq free tier with no credit card required.

Embedding model: `paraphrase-multilingual-MiniLM-L12-v2` — runs locally inside the RAG Docker container. Zero API cost. Supports English and Bangla with 768-dimensional vectors. HNSW-indexed in pgvector.

Appendix C

Repository Structure

```
csedu-platform/
|-- frontend/                    # Next.js
|   |-- Dockerfile
|   |-- src/
|   |   |-- app/                 # App router pages
|   |   |-- components/          # Reusable components
|   |   `-- lib/                 # API client, auth helpers
|   `-- package.json
|
|-- api/                         # Go monolith
|   |-- Dockerfile
|   |-- Dockerfile.worker        # Ingestion worker entrypoint
|   |-- Dockerfile.fineworker    # Fine cron entrypoint
|   |-- cmd/
|   |   |-- api/main.go
|   |   |-- ingestion-worker/main.go
|   |   `-- fine-worker/main.go
|   `-- internal/
|       |-- auth/                # JWT, SSO, middleware
|       |-- archive/             # Digital archive module
|       |-- library/             # Loans, fines, catalog
|       |-- research/            # Research repository
|       |-- projects/            # Student showcase
|       |-- ai/                  # RAG proxy, AI admin
|       `-- db/                  # PostgreSQL client
|
|-- rag/                         # Python FastAPI RAG service
|   |-- Dockerfile
|   |-- main.py                  # FastAPI app
|   |-- embedder.py              # sentence-transformers wrapper
|   |-- retriever.py             # pgvector similarity search
|   |-- llm_client.py            # Groq API client (tiered)
|   `-- requirements.txt
|
|-- infra/
|   |-- db/
```

```
|   |   `-- init.sql            # Schema + pgvector extension
|   |-- nginx/
|   |   `-- nginx.conf
|   `-- prometheus/
|       `-- prometheus.yml
|
|-- .env.example                # Committed with placeholders
|-- .gitignore                  # .env must be here
|-- docker-compose.yml
`-- README.md
```

Listing C.1: Recommended repository layout